Aula 06 — Protocolo HTTP

Arquitetura REST

Introdução ao Express

Aula 06 | Programação Back-end 2

Do catálogo no terminal para uma API HTTP

Missão da aula

Observar método, caminho, status, headers e body

Criar um servidor Express no Sandbox

Transformar a api-produtos cumulativa em uma API HTTP

Preservar regras e persistência já construídas

Fundamentos essenciais

HTTP e REST

Cliente e servidor trocam mensagens. A URL identifica recursos e cada requisição é compreendida sem conversa oculta.

Métodos e status

GET consulta. POST cria. Respostas 2xx indicam sucesso, 4xx problema do cliente e 5xx falha inesperada.

SENAI

Express no Sandbox

Experimentos pequenos antes da API cumulativa

Preparação do Exemplo00-Sandbox

```
node --version
mkdir exemplos\Exemplo00-Sandbox
cd exemplos\Exemplo00-Sandbox
npm init -y
npm install express
```

Mini-desafio 1 — servidor e saúde (server.js)

SENAI

```
import express from 'express';

const app = express();

app.get('/saude', (_req, res) => {
  res.status(200).json({ status: 'ok' });
});
```

Mini-desafio 1 — escutar a porta (server.js)

```
app.listen(3100, '127.0.0.1', () => {
  console.log('Servidor em http://127.0.0.1:3100');
});

GET http://127.0.0.1:3100/saude
GET http://127.0.0.1:3100/ausente
```

Mini-desafio 2 — GET não altera

```
const produtos = [{ id: 1, nome: 'Teclado' }];

app.get('/produtos', (_req, res) => {
  res.status(200).json({ dados: produtos });
});

GET /produtos  → quantidade 1
GET /produtos  → quantidade 1
```

Mini-desafio 3 — interpretar JSON

```
app.use(express.json());

// Registre antes das rotas
// O body JSON ficará disponível em req.body
```

Mini-desafio 3 — criar com POST

```
app.post('/produtos', (req, res) => {
  if (typeof req.body.nome !== 'string' ||
      req.body.nome.trim() === '') {
    return res.status(400).json({
      erro: 'Nome é obrigatório.'
    });
  }
```

Mini-desafio 3 — responder 201

```
  const produto = {
    id: produtos.length + 1,
    nome: req.body.nome.trim()
  };
  produtos.push(produto);
  res.status(201).json({ dados: produto });
});
```

Teste rápido — escolher o status

```
200  consulta concluída
201  produto criado
400  entrada inválida
404  caminho sem rota
500  falha inesperada
```

Mini-desafio 4 — modelar recursos

```
GET  /api/produtos
POST /api/produtos

O caminho identifica a coleção.
O método comunica consulta ou criação.
```

SENAI

Evolução hands-on

A mesma api-produtos agora recebe HTTP

Passo 1 — instalar e preparar

```
cd ..\api-produtos

npm install express

package.json
"start": "node server.js"
```

Passo 2 — remover o CLI

```
Abra app.js

Apague a função executar()
Apague a chamada executar()
Remova o fluxo que interpreta process.argv

A entrada agora será HTTP
```

app.js — imports e configuração

```
import express from 'express';
import { resolve } from 'node:path';
import { criarCatalogoArquivo } from
  './catalogo/catalogoArquivo.js';
import { carregarAmbiente } from
  './config/ambiente.js';
```

app.js — reutilizar o catálogo

```
const configuracao =
  carregarAmbiente('.env');
const caminhoCatalogo =
  process.env.CATALOGO_ARQUIVO ||
  resolve(import.meta.dirname,
    'data/produtos.json');
const catalogo =
  criarCatalogoArquivo(caminhoCatalogo);
```

app.js — criar a aplicação Express

```
export const app = express();
app.use(express.json());

// Rota de saúde
app.get('/api/saude', (_req, res) => {
  res.status(200).json({
    status: 'ok',
    ambiente: configuracao.ambiente
  });
});
```

Passo 3 — GET substitui listar (app.js)

```
app.get('/api/produtos',
  async (_req, res) => {
  try {
    const produtos = await catalogo.listar();
    res.status(200).json({
      sucesso: true, dados: produtos
    });
  } catch (erro) {
```

GET — tratar falha inesperada

```
    res.status(500).json({
      sucesso: false,
      erro: 'Falha interna no servidor.'
    });
  }
});
```

POST substitui criar

```
app.post('/api/produtos',
  async (req, res) => {
  try {
    const { nome, preco, estoque, categoria } =
      req.body;
    const produto = await catalogo.criar({
      nome, preco, estoque, categoria
    });
```

POST — responder criação ou validação

```
    res.status(201).json({
      sucesso: true, dados: produto
    });
  } catch (erro) {
    res.status(400).json({
      sucesso: false, erro: erro.message
    });
  }
});
```

A tradução do CLI para HTTP

SENAI

Antes: terminal

listar e criar eram lidos em process.argv. O resultado aparecia com console.log ou console.table.

Agora: requisição

GET e POST chegam por URL e body. O resultado sai com status e JSON.

Passo 4 — criar server.js

```
import { app } from './app.js';
import { carregarAmbiente } from
  './config/ambiente.js';

const config = carregarAmbiente('.env');
const porta = config.porta || 3000;
```

server.js — começar a escutar

```
app.listen(porta, () => {
  console.log(
    `Servidor rodando em http://localhost:${porta}`
  );
});
```

Passo 5 — testar no Postman

Evidências manuais da nova API HTTP

Iniciar e verificar a saúde

```
npm start

GET http://localhost:3000/api/saude

Esperado: status 200
{ "status": "ok" }
```

Listar o catálogo persistente

```
GET http://localhost:3000/api/produtos

Esperado: status 200

Os produtos aparecem dentro de dados
```

Criar o Monitor com POST

```
POST http://localhost:3000/api/produtos

{
  "nome": "Monitor",
  "preco": 1200,
  "estoque": 2,
  "categoria": "Vídeo"
}
```

Comprovar a persistência

```
GET http://localhost:3000/api/produtos

Monitor aparece na listagem

Ctrl+C
npm start
GET /api/produtos

Monitor continua no catálogo
```

Diagnóstico prático

```
req.body indefinido  → express.json antes da rota
criação responde 200 → usar 201
processo encerra      → conferir app.listen
porta ocupada         → encerrar com Ctrl+C
erro expõe caminho    → mensagem pública estável
```

Fechamento prático

```
Mostre no Postman:

GET saúde          → 200
GET produtos       → 200
POST produto       → 201
POST inválido      → 400
rota ausente       → 404
```